6-4 · CDC(Debezium) 실시간 캡처 · 정리·회고 0.5H

정리·회고

REST API 를 한 장으로, 6-5 적재 준비 확인

이 시간의 목표

30분 뒤에는

① Connect REST API 여덟 명령을 한 장으로 정리한다

② 체크리스트를 스스로 점검한다

③ 자주 겪는 오류 네 가지의 원인과 해결을 안다

④ 6-5 를 위해 커넥터·토픽이 살아 있는지 확인한다

오늘의 compose 스택은 내리지 않습니다 — 6-5 가 그 위에서 이어집니다

오늘의 요약

Connect REST API 여덟 명령 — 한 장으로

하려는 일	명령
플러그인 목록	GET /connector-plugins
커넥터 등록	POST /connectors + JSON 본문
커넥터 목록	GET /connectors
상태 확인	GET /connectors/이름/status
설정 보기	GET /connectors/이름/config
설정 수정	PUT /connectors/이름/config
재시작	POST /connectors/이름/restart
삭제	DELETE /connectors/이름

오늘 커넥터 이름은 mysql-cdc — 삭제 뒤 재등록이 오류 복구의 기본 절차

오늘의 체크리스트

여덟 줄로 스스로 확인해 보세요

1. 폴링 대비 CDC 의 장점 2가지 · DELETE 를 못 잡는 이유를 설명했다
2. log_bin=ON, binlog_format=ROW 를 직접 확인했다
3. 스냅숏과 스트리밍의 순서를 설명할 수 있다
4. 커넥터를 REST API 로 등록하고 RUNNING 을 확인했다
5. 토픽 이름 규칙(topic.prefix.DB.테이블)을 설명했다
6. op=r 스냅숏과 c/u/d 의 before/after 를 대조했다
7. 파이썬으로 읽고, 평범한 SQL 만으로 이벤트가 흘러나옴을 봤다
8. CDC 는 최소 한 번 전달 — 받는 쪽이 멱등해야 한다는 것을 안다

자주 겪는 오류

증상으로 찾는 원인과 해결

증상	원인	해결
status FAILED **접속·권한 오류**	hostname 이 localhost REPLICATION 권한 없음	trace 첫 줄 → 바로잡고 DELETE 뒤 재등록
RUNNING 인데 **이벤트가 안 온다**	binlog 꺼짐 · ROW 아님 include.list 밖의 DB	SHOW VARIABLES 확인 my.cnf ROW 로 재기동
Connect 가 죽거나 **브로커를 못 찾음**	BOOTSTRAP_SERVERS 가 localhost:9092	compose·JSON 모두 kafka:29092 로
3306 충돌로 **mysql 이 안 뜸**	2주차 de-mysql 이 살아 있음	docker stop de-mysql 또는 3307:3306

컨테이너 안에서 localhost 는 자기 자신 — 호스트명은 mysql · 브로커는 kafka:29092(6-2 리스너 2계층)

예상 질문

자주 나오는 질문 네 가지

1. **테이블이 100개면 토픽도 100개인가요?**
   기본은 그렇다 — 라우팅 설정으로 합칠 수도 있지만 테이블별이 스키마 관리에 유리

2. **스냅숏이 오래 걸리면?**
   큰 테이블이면 몇 시간 — snapshot.mode 로 건너뛰거나 증분 스냅숏을 쓴다

3. **원본 DB 에 부담이 없나요?**
   폴링보다 훨씬 적다 — binlog 를 읽는 I/O 와 스냅숏 중 전체 조회 부담은 있다

4. **PostgreSQL 도 되나요?**
   된다 — WAL 을 읽는 커넥터가 있고 개념은 같다 · MongoDB·Oracle 용도 있다

6-5 를 위한 준비 — 스택은 내리지 않는다

1. **docker compose ps**
   kafka · mysql · connect 세 서비스가 떠 있다

2. **curl -s localhost:8083/connectors**
   목록에 mysql-cdc 가 있다

3. **kafka-get-offsets.sh --topic shop.shop.orders**
   이벤트가 쌓여 있으면 준비 완료 — 6-5 는 이 토픽을 MinIO 로 흘려보낸다

심화(선택)

transforms

unwrap

다음 회차 예고

6-5 · Kafka Connect 와 MinIO 적재

01
이론 1H
Sink 구조 · S3 API
MinIO · flush.size

02
실습 2.5H
S3 Sink 로
MinIO 적재·검증

03
정리 0.5H
주차 산출물
커넥터 JSON·토픽

04
사전 준비
S3 Sink zip 을
week6/plugins/ 에

6-4 스택(kafka·mysql·connect)이 떠 있고 mysql-cdc 가 RUNNING 이어야 합니다 — 플러그인은 강의장 네트워크가 느릴 수 있으니 수업 전에 받아 두세요

6-4 · 정리·회고 0.5H 마무리

다음 회차는 6-5 MinIO 적재

S3 Sink 커넥터로 CDC 토픽을 MinIO 에 실시간 적재해 MySQL → Debezium → Kafka → MinIO 를 완성합니다

- ✓ 소스 vs 싱크 커넥터 역할 차이
- ✓ MinIO 버킷 만들기(S3 API 호환)
- ✓ store.url 로 MinIO 를 가리키는 Sink 설정
- ✓ DB 변경 → MinIO 파일 엔드투엔드 검증

준비할 것

스택 유지 · S3 Sink 플러그인 zip